# Experiment No. 3

*Time-Series Analysis and Visualization of IoT Sensor Data*

## 1. Aim

To simulate simple IoT sensor readings in software, and to learn the basic steps of analyzing and visualizing time-series data using Python: cleaning the data, smoothing it, spotting unusual readings, and plotting it.

## 2. Objectives

- Understand what makes sensor data a “time series” (readings taken one after another over time).
- Generate simple, realistic sensor data (temperature, humidity, air quality) without needing any hardware.
- Handle missing sensor readings — a common real-world problem.
- Learn two basic ways to reduce noise in data: resampling and rolling average.
- Learn a simple way to detect unusual/abnormal readings.
- Visualize sensor data using easy-to-read charts.

## 3. Theory

An IoT sensor takes a reading again and again — say, once every 15 minutes — and sends it, with a timestamp, to a server or cloud platform. This ordered sequence of time-stamped values is called a time series; the order and spacing in time actually matters.

**Common patterns seen in sensor data:**

- Trend — data slowly rising or falling over a long period.
- Seasonality (daily cycle) — a pattern that repeats regularly, e.g. temperature rising in the afternoon and falling at night.
- Noise — small random ups and downs from sensor imperfection.
- Missing data — a reading that never arrives, due to a network drop or sleep mode.
- Anomalies — sudden, unusual spikes or drops — could be a fault or something genuinely important.

**A few simple techniques make raw, noisy sensor data more useful:**

- Resampling — group readings into bigger time buckets (e.g. 15-min → hourly) so data is smaller and smoother.
- Rolling average — average the last few readings as you move through the data, smoothing short-term noise while keeping the trend.
- Anomaly detection — flag any reading far from the average so it stands out.
- Correlation — check whether two sensors tend to move together.

## 4. Software / Tools Required

Python 3.x · NumPy, Pandas, Matplotlib · Jupyter Notebook / Google Colab / any Python IDE. No physical sensor or microcontroller is needed — sensor-like data is generated entirely in Python.

# 5. Flowchart

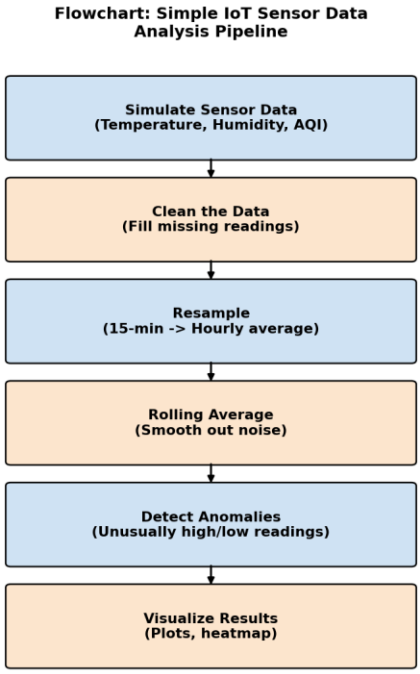



*Fig. 5.1 — Simple pipeline: simulate data → clean it → resample → smooth → detect anomalies → visualize.*

# 6. Step-by-Step Procedure

## Step 1 — Simulate the Sensor Data

A repeating daily wave (mimicking day/night temperature change) plus random noise (mimicking real sensor imperfection) is used to generate a reading every 15 minutes for 10 days. A few sudden temperature spikes and missing readings are added deliberately, so the dataset behaves like a real, slightly imperfect sensor feed.

```
timestamps = pd.date_range("2026-07-20", periods=periods, freq="15min")
daily_cycle = 5 * np.sin(2*np.pi*t/period - np.pi/2)   # day/night wave

temperature = 27 + daily_cycle + np.random.normal(0, 0.6, periods)
humidity = 65 - 1.5*daily_cycle + np.random.normal(0, 2.5, periods)
aqi = 80 + np.random.normal(0, 6, periods)

# add a few spikes and a few missing readings to simulate a real feed
```

## Step 2 — Clean the Data

Missing values are filled using time-based interpolation — estimating a sensible value from the readings just before and after the gap.

```
df["temperature_C"] = df["temperature_C"].interpolate(method="time")
```

## Step 3 — Resample (Hourly Averages)

Grouping all readings within each hour and averaging them gives a smoother, easier-to-read hourly series.

```
hourly = df.resample("h").mean()
```

### Step 4 — Rolling Average (Smoothing)

A rolling average looks at the last 16 readings (4 hours) at every point and averages them, smoothing short-term noise while keeping the overall rise-and-fall pattern.

```
df["temp_roll_mean"] = df["temperature_C"].rolling(window=16).mean()
```

### Step 5 — Detect Simple Anomalies

A reading is flagged as an anomaly if it is more than 2.5 standard deviations away from the mean temperature.

```
mean_temp = df["temperature_C"].mean()
std_temp = df["temperature_C"].std()
df["is_anomaly"] = (df["temperature_C"] - mean_temp).abs() > 2.5 * std_temp
```

### Step 6 — Visualize the Results

The data is plotted in four ways: raw readings, the smoothed trend with anomalies marked, the hourly-resampled series, and a correlation heatmap between the three sensors.

## 7. Results and Observations

### 7.1 Raw Sensor Readings

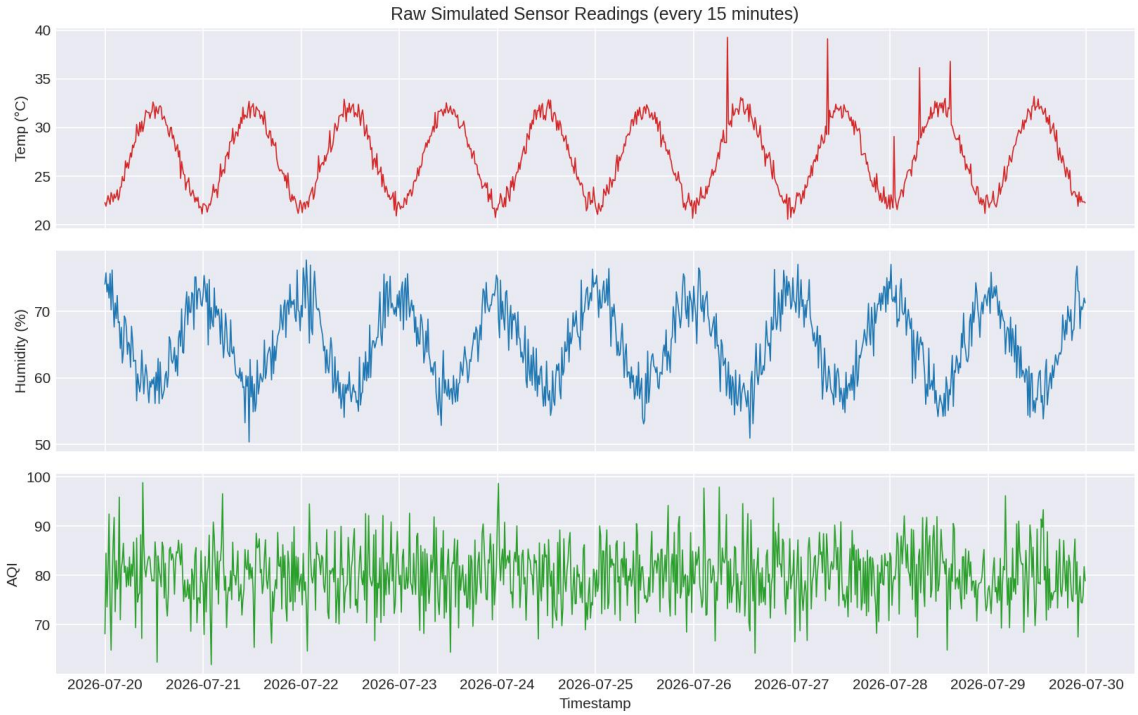



*Fig. 7.1 — Raw simulated readings for temperature, humidity, and AQI over 10 days.*

Temperature clearly rises and falls once a day (the daily cycle). Humidity moves opposite to temperature, just as designed.

## 7.2 Smoothed Trend and Anomalies

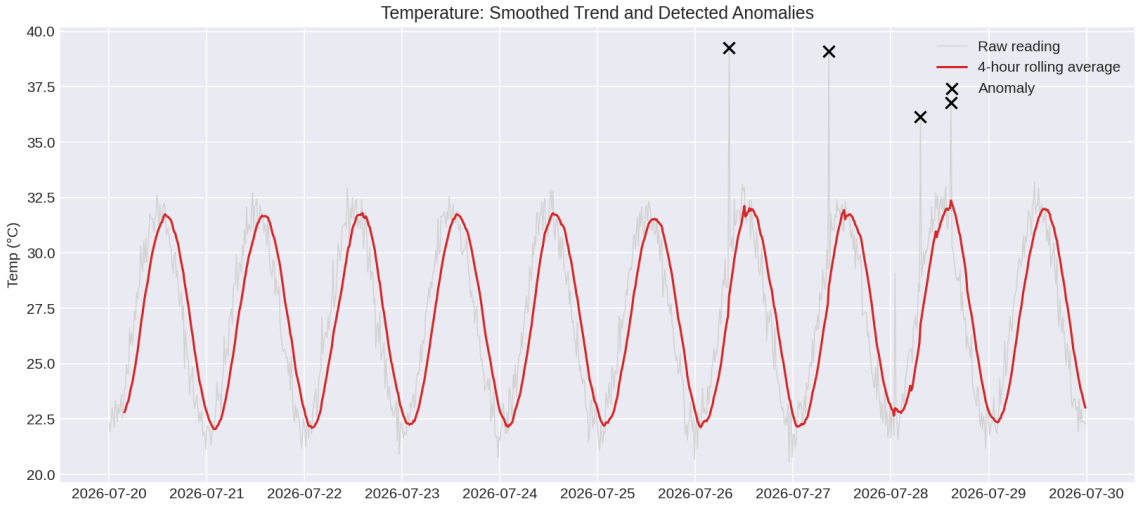



*Fig. 7.2 — Grey: raw temperature. Red: 4-hour rolling average. Black ×: detected anomalies.*

The rolling average line is much smoother than the raw data and makes the daily pattern obvious. The black × marks show where the simple anomaly check correctly caught the spikes that were added.

## 7.3 Hourly-Resampled Temperature

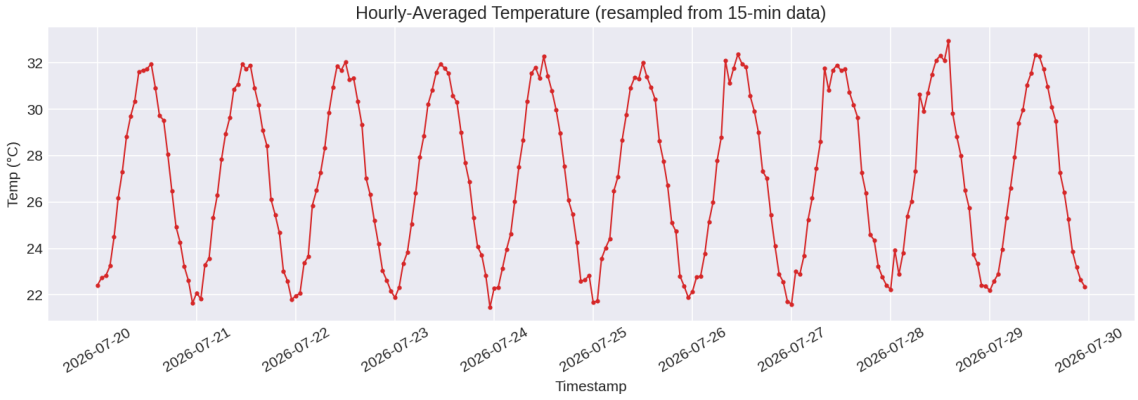



*Fig. 7.3 — Hourly-averaged temperature over the full 10-day period.*

Compared to the raw 15-minute plot, this hourly view is easier to read while still showing the same overall pattern and trend.

### 7.4 Correlation Between Sensors

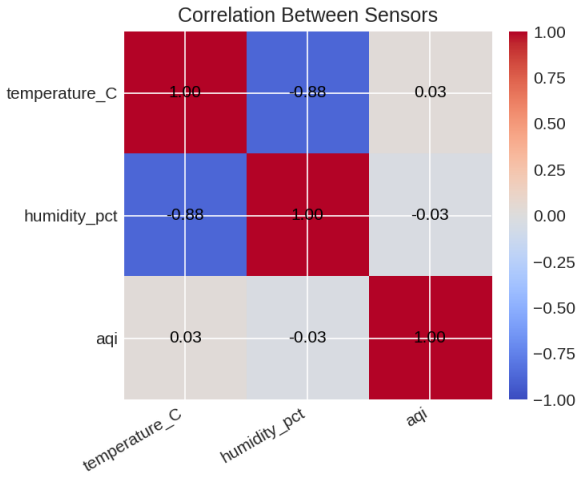



*Fig. 7.4 — Correlation heatmap between temperature, humidity, and AQI.*

Temperature and humidity show a strong negative correlation (around −0.88) — when one goes up, the other tends to go down. AQI shows almost no relationship with either, since it was generated independently.

## 8. Output

```
Dataset shape: (960, 3)
                     temperature_C  humidity_pct     aqi
timestamp
2026-07-20 00:00:00      22.30         74.11        68.12
2026-07-20 00:15:00      21.93         75.81        84.49
2026-07-20 00:30:00      22.43         72.93        73.56
2026-07-20 00:45:00      23.01         74.13        81.44
2026-07-20 01:00:00       NaN          72.02        92.44

Missing temperature values filled: 20 -> 0
Anomalies detected: 4

Correlation matrix:
               temperature_C  humidity_pct     aqi
temperature_C       1.00         -0.88         0.03
humidity_pct       -0.88          1.00        -0.03
aqi                 0.03         -0.03         1.00
```

## 9. Conclusion

Using only Python and simulated data, this practical covered the basic building blocks of IoT time-series analysis: generating sensor-like data, filling in missing readings, smoothing noisy data through resampling and rolling averages, spotting unusual readings, and visualizing everything clearly. The same steps apply directly to real sensor data — only Step 1 would change, from generating data to reading it from an actual sensor or cloud platform.